ASD Group 18 Technical Report

https://github.com/rigels51/ASD-Project-Group18

1.0 Project Overview

1.1 Project Overview and Objectives

The project is a University Management System developed as a collection of independent microservices. Each group member was responsible for developing an individual university management feature while following a shared architecture and frontend design.

The main objective of the project was to develop a functional integrated application that supports common university management tasks such as managing student records, staff, timetables, courses and enrolments. Each feature was designed with separate frontend, backend/API and database responsibilities to maintain modularity and make the system easier to integrate.

Another major objective was to incorporate local AI functionality into the individual features. The AI Mode allows users to ask natural-language questions about application data, with current database information provided as context to reduce unsupported responses.

Beyond AI Mode, each feature also provides an MCP Mode for structured, tool-based queries against live data and a RAG Mode for citation-backed, retrieval-grounded answers, both powered by shared MCP and RAG servers

The project also aimed to demonstrate modern software development practices, including Agile development, GitHub-based collaboration, Docker containerisation, REST APIs, automated GitHub Actions workflows and the Plan → Act → Observe → Adapt agentic workflow.

1.2 Team Members

Student 1: Rigel 25876487

Student 2: Jeriko Arceo 24806008

Student 3: Lazizbek Ismoilov 14567426

Student 4: Yi Zhang 25402140

Student 5: Vu Tien Thanh Nguyen 14673154

1.3 Individual Feature Allocation (i)

Student 1: Rigel Rivamonte

This feature manages university student records. Users can view, search, create, update, and delete student information. Each record includes a student ID, name, course, year level, email, phone number, GPA, and academic status.

Release 2 extends the Release 1 student-records application by adding a Multi-Agent workflow to the Student 1 frontend and backend. The integration connects the student feature to the shared, non-containerised Multi-Agent Server and supports running a workflow, viewing workflow details and history, checking status, and submitting a human decision. The existing AI-Mode, MCP, and RAG flows remain part of the application.

Student 2: Jeriko Arceo

Feature: Staff Management

This feature is responsible for managing staff records for the University management application. The feature allows the user to view, create, edit, and delete staff member details from the university. It also allows staff to be assigned to classes.

Its main functions include:

- Create staff
- Read staff
- Update staff
- Delete staff
- AI agent to assist with assigning staff
- Get staff data
- Get subject data
- Assign staff to classes

Student 3: Lazizbek Ismoilov

Feature: Timetable and Classes Management

This feature manages when and where each course/subject session runs. Students and lecturers can browse and filter the timetable; admin users can create, update, and delete sessions; the system automatically detects room/time scheduling clashes; and a local AI agent answers natural-language scheduling questions grounded in the live timetable data.

- Its main functions include:
- Create a new class session (course code, day, time, room, semester)
- Read / browse all sessions and filter by course code
- Update an existing session's room or time
- Delete a session
- Automatically detect scheduling clashes (same day, room, overlapping time)
- AI agent to answer natural-language scheduling questions, grounded in real timetable data
- Role-based views for Student, Lecturer, and Admin
- Get sessions by course code (cross-feature lookup point for Course Catalogue)

For Release 2, the Timetable feature was connected to the shared Multi-Agent Server. The Timetable backend registers the team's Multi-Agent blueprint with DOMAIN=timetable, which exposes /multi-agent/run, /multi-agent/decision, /multi-agent/workflow/<id>, /multi-agent/history and /multi-agent/health. A new Multi-Agent tab in the frontend shows the Planner, Worker and Reviewer output, including the Reviewer's risk level and six automatic checks. It also has the human review form (Accept, Partially Accept, Correct, Reject) and the Timetable workflow history. The MULTI_AGENT_ENABLED flag switches the feature off in CI and in the cloud, where the routes return 503. I also added post-commit endpoint tests and fixed a pre-commit security finding. Beyond my own feature, I built the team's cloud deployment on Azure Container Apps (deploy_azure.py and cloud-deployment.yml) and the Cloud Deployment Review mode of the shared agentic loop.

Student4: Yi Zhang

Feature: Course and Enrollment Management

For Release 2, my responsibility was to add Multi-Agent functionality, automated testing, and cloud deployment support to the Course and Enrollment Management system.

The Multi-Agent system uses three agents: Planner, Worker, and Reviewer. It also supports Human Review, allowing users to check results and make final decisions.

For testing, I used Ruff, detect-secrets, and pip-audit for pre-commit security checks. I also added automated API tests using Python unittest and GitHub Actions.

For cloud deployment, our application was deployed to Microsoft Azure. The normal course management functions remain available, while AI features are disabled by default in the cloud.

Student 5: Vu Tien Thanh Nguyen

Feature: Assessment & Grades Management

My responsibility was to design and implement the Assessment & Grades Management feature of the University Management System. This feature manages course assessments and student results. It allows users to view, search, create, update, and delete assessment and grade records.

The feature also includes an AI Mode that allows users to ask natural-language questions about assessments, grades, and student performance. The AI retrieves current assessment and grade information before generating a response using the Plan → Act → Observe → Adapt workflow.

The main functions include:

- View and search assessments by course, assessment type, or keyword
- View individual assessment details
- Create new assessments
- Update existing assessments
- Delete assessments
- View student grade records
- Record new student grades
- Update and delete grade records
- Search grades by student ID or course
- Validate student IDs using the Student Records Management service
- Retrieve course information from the Course Management service
- Use AI Mode to answer questions about assessments, grades, and student performance
- Use current database information to provide grounded AI responses

The feature was implemented as three separate microservices: an Nginx frontend service, Flask backend/API service, and Flask/SQLite database service. Docker is used to containerise each service, while Ollama with Qwen is used for the local AI functionality.

For Release 1, the Assessment & Grades Management feature was extended with MCP Mode and RAG Mode using the shared project-level MCP and RAG servers. MCP Mode provides read-only access to assessment and grade information through three predefined tools: list_course_assessments, get_student_grade_summary, and get_upcoming_assessments. RAG

Mode provides retrieval-grounded answers using assessment records, anonymised course-level grade statistics, and the assessment policy document. RAG responses include citations, source information, and confidence levels, and the system returns insufficient evidence when the retrieved information does not support the question.

2.0 Final Repository Structure

2.1 Shared GitHub Repository Layout

The repository (ASD-Project-Group18) is organised, so each student's feature is fully self-contained, with shared, team-wide resources kept separate from individual work.

Top-level layout:

- .github/workflows/ — one CI/CD workflow file per student (e.g. student-3.yml), each scoped to only trigger on changes to that student's own folder
- shared/ — the unified home page (homepage.html), shared CSS theme, and its Dockerfile
- student-1/ through student-5/ — one folder per student, each containing that student's frontend, backend/API, and database code
- docker-compose.yml — the single shared file orchestrating all services across all five features plus the shared home page
- README.md — project overview and per-feature setup instructions
- ai-services/mcp-server/
- ai-services/rag-server/

Each student-N/ folder follows a three-tier internal structure, though the exact naming varies slightly by student (e.g. frontend/backend/database for Student 3 and Student 5, versus frontend-service/enrolment-service/database-service for Students 1, 2, and 4) — reflecting that folder-level naming was standardised to student-1…student-5 as a group during Release 0, while internal sub-folder naming was left to each student's own convention.

2.2 Branching Strategy and Folder Organization

Each student develops on their own feature branch (e.g. student-3/timetable-integration, jeriko, Yi-Zhang, rigel) and merges into main via Pull Request, following the spec's required PR-based workflow rather than direct commits to main.

Folder organisation directly enforces the microservice boundary rule: a student's code only ever lives inside their own student-N/ folder, and cross-feature data is accessed only via that feature's REST API, never by reading another student's database file or importing their code directly.

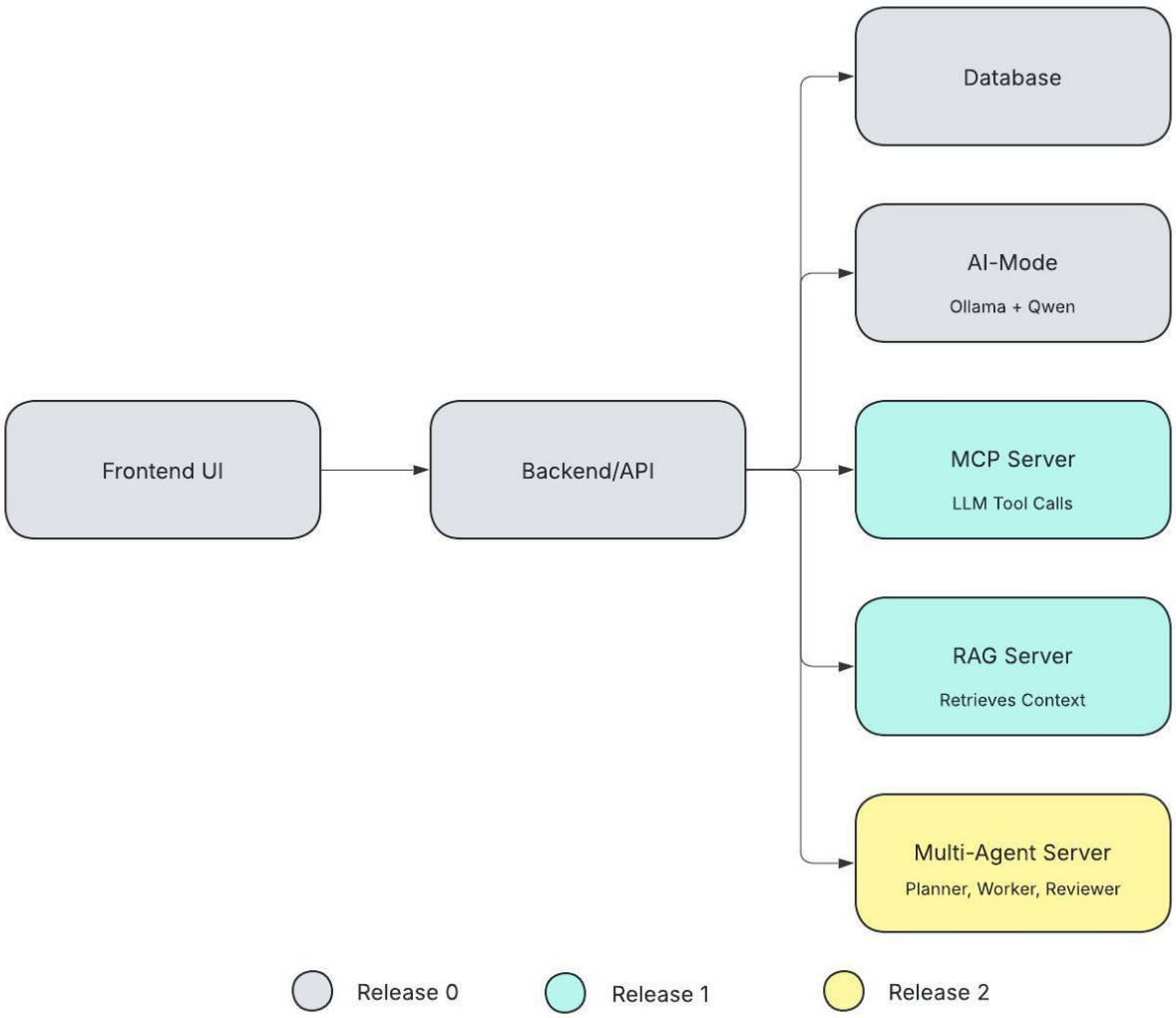
During Release 0, the team encountered and resolved two structural issues through this branching workflow:

- Inconsistent top-level naming (student1, Student2, Student4, student-5, student-3) was standardised to student-1 through student-5 in a coordinated update to main
- A real merge conflict occurred in the shared docker-compose.yml when two students' service additions landed close together; it was resolved by merging both services' blocks rather than discarding either side's work

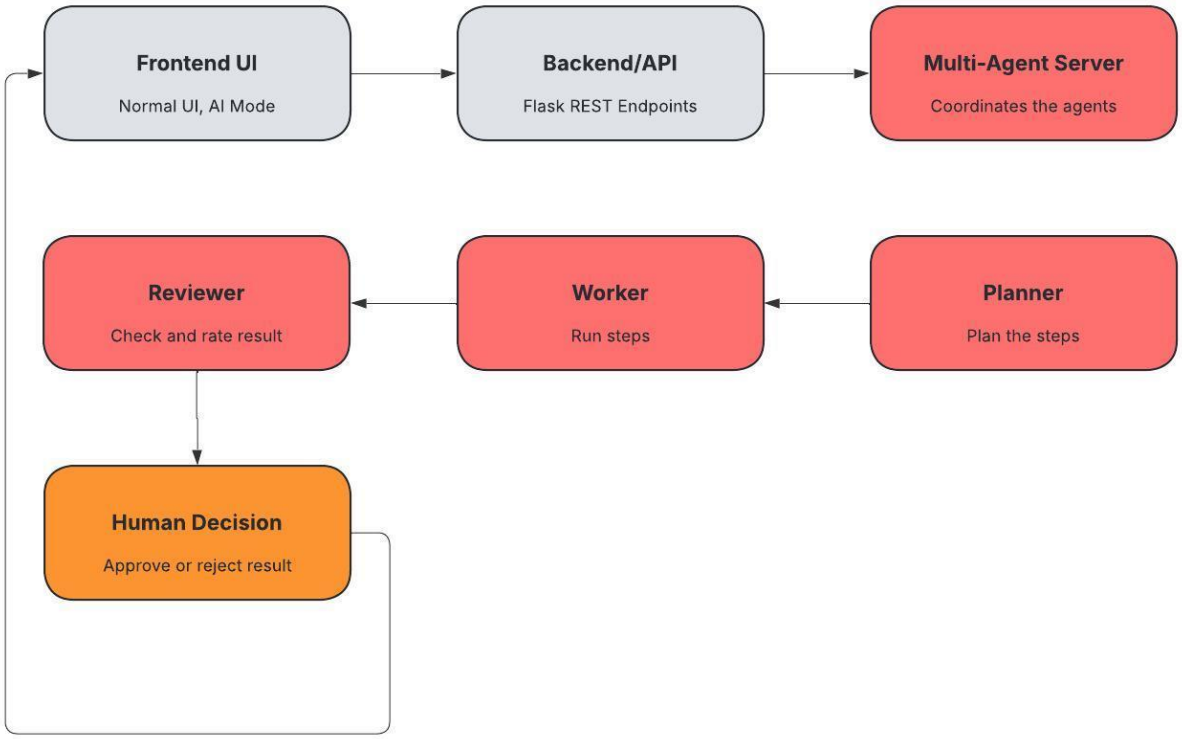
Commits are made in small, regular increments and pushed early to reduce the size and difficulty of merge conflicts, consistent with the mitigation strategy documented in each student's Risk Management Plan (Section 2.4).

3.0 Release 2 Design Flow

3.1 Update Design Flow



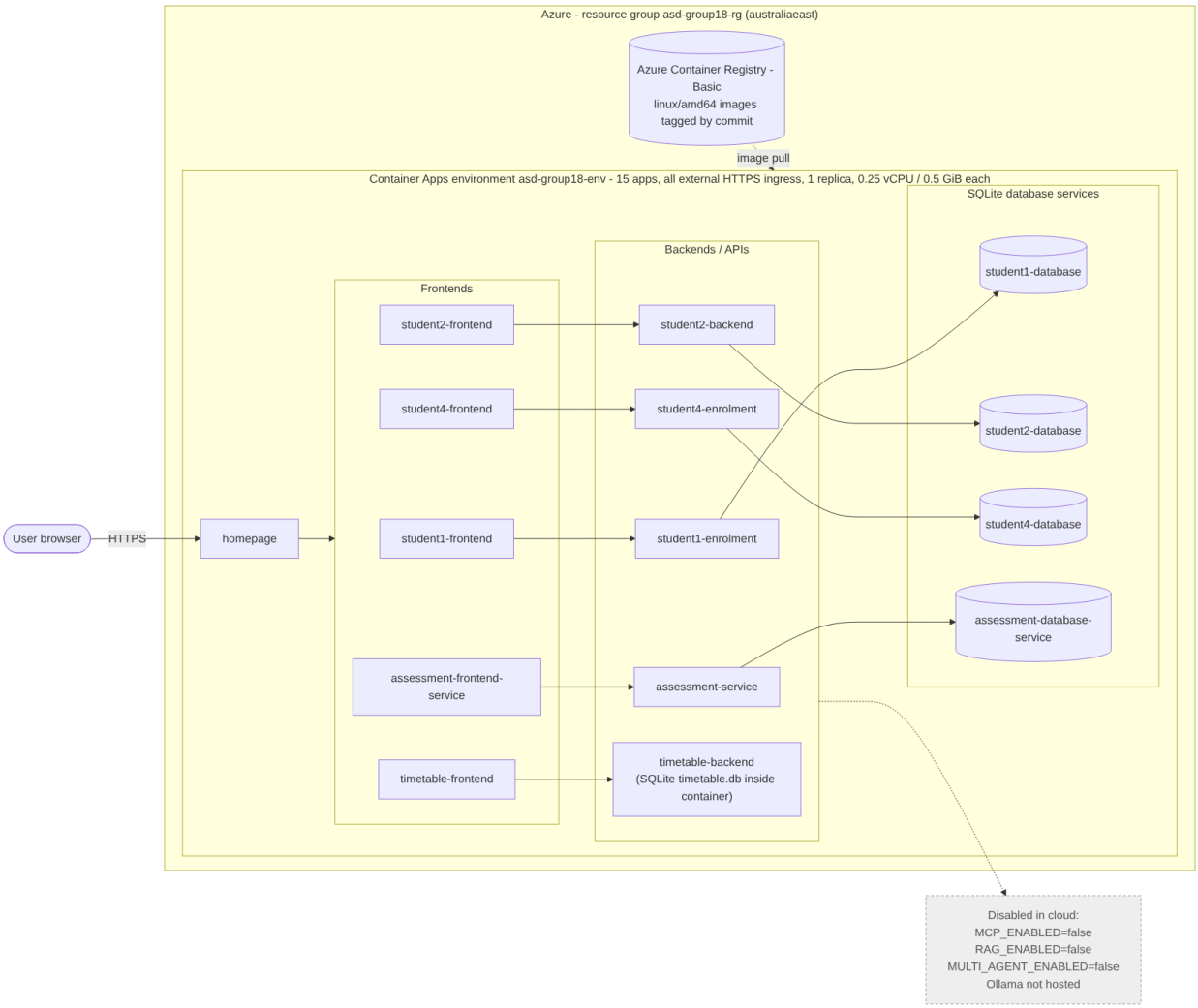
3.2 Multi-Agent Flow



4.0 Cloud Deployment and CI/CD Architecture

4.1 Cloud deployment architecture

The integrated application runs on Azure Container Apps in one environment (asd-group18-env) inside resource group asd-group18-rg (Australia East). Each of the 15 Docker Compose services becomes one Container App with the same name, running one replica at 0.25 vCPU / 0.5 GiB. The Timetable database is a SQLite file inside timetable-backend; the other four features run a separate database service. Images are built for linux/amd64 and pushed to an Azure Container Registry (Basic), tagged with the commit SHA. All 15 apps have public HTTPS addresses (external ingress). At build time, the deploy script rewrites the frontends' http://localhost:<port> API URLs in a build copy to point at the cloud URLs, so the repository files stay unchanged and local Docker Compose keeps working. Every backend runs with MCP_ENABLED, RAG_ENABLED and MULTI_AGENT_ENABLED set to false.



4.2 DevOps pipeline architecture

A commit first passes the pre-commit hooks. It is then pushed to a feature branch and merged to main by pull request. Each merge runs that student's workflow. When a student workflow finishes, or when any team member clicks **Run workflow**, cloud-deployment.yml starts. Deployment only goes ahead if the latest run of all five student workflows on main is green.



4.3 Description of cloud-deployment.yml

- **Triggers:** workflow_run (completion of student-1.yml to student-5.yml on main) and workflow_dispatch, so any team member can run it.
- **Job 1, ci-gate:** looks up the latest run of each student workflow on main and writes a status table to the run summary. If any run is not success, it stops the deployment.
- **Job 2, deploy** (needs: ci-gate): checks out the exact commit CI validated, logs in to Azure with azure/loginusing GitHub OIDC and a user-assigned managed identity (secrets AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID; no password stored), sets up Docker Buildx, runs python3 scripts/deploy/deploy_azure.py deploy, then publishes deployment.json to the run summary and as an artifact.

4.4 Description of student-3.yml (Release 2 changes)

It extends the Release 1 workflow with a pytest endpoint stage. That stage runs the post-commit tests against an isolated database, uploads the JUnit report as the student-3-test-reports artifact and writes a results table to the run summary. The workflow also runs a live create-then-detect clash check against the running container. It sets MCP, RAG and Multi-Agent to false, so CI needs no local AI servers. It still validates the 10-record database seed, the core endpoints and both Docker im

5.0 Pre-Commit Security Testing

Rigel Rivamonte

docs > release-2 > testing > student-1-precommit.txt

```
# Student 1 - Rigel Rivamonte - pre-commit security scan
# date: 2026-10-22T10:05:55+00:00
# commit: d814dcb
# scope: student-1/ (28 tracked files)
# hooks: ruff-security, detect-secrets, pip-audit (.pre-commit-config.yaml)
# exit_code: 0

Ruff security rules (flake8-bandit S).....Passed
detect-secrets (no new credentials).....Passed
pip-audit (known-vulnerable dependencies).....Passed
```

Jeriko Arceo



```
# Student 2 - Jeriko Arceo - pre-commit security scan
# date: 2026-10-21T13:26:16+00:00
# commit: 3ef8d12
# scope: student-2/ (27 tracked files)
# hooks: ruff-security, detect-secrets, pip-audit (.pre-commit-config.yaml)
# exit_code: 0

Ruff security rules (flake8-bandit S).....Passed
detect-secrets (no new credentials).....Passed
pip-audit (known-vulnerable dependencies).....Passed
```

Lazizbek Ismoilov

```
Ruff security rules (flake8-bandit S).....Passed
detect-secrets (no new credentials).....Passed
pip-audit (known-vulnerable dependencies).....Passed
(asd-precommit) (base) laz@Lazizbeks-MacBook-Air ASD-Project-Group18 %
```

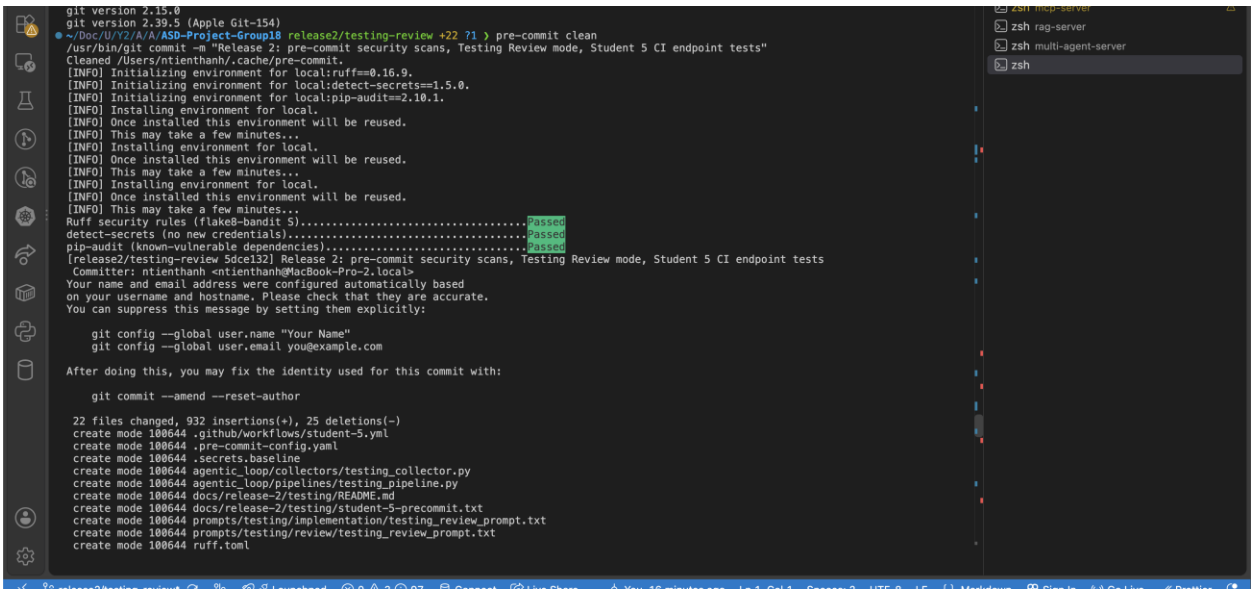
Yi Zhang

docs > release-2 > testing > student-4-precommit.txt

```
# Student 4 - Yi Zhang - pre-commit security scan
# date: 2026-10-21T19:59:16+00:00
# commit: ada8f8a
# scope: student-4/ (32 tracked files)
# hooks: ruff-security, detect-secrets, pip-audit (.pre-commit-config.yaml)
# exit_code: 0

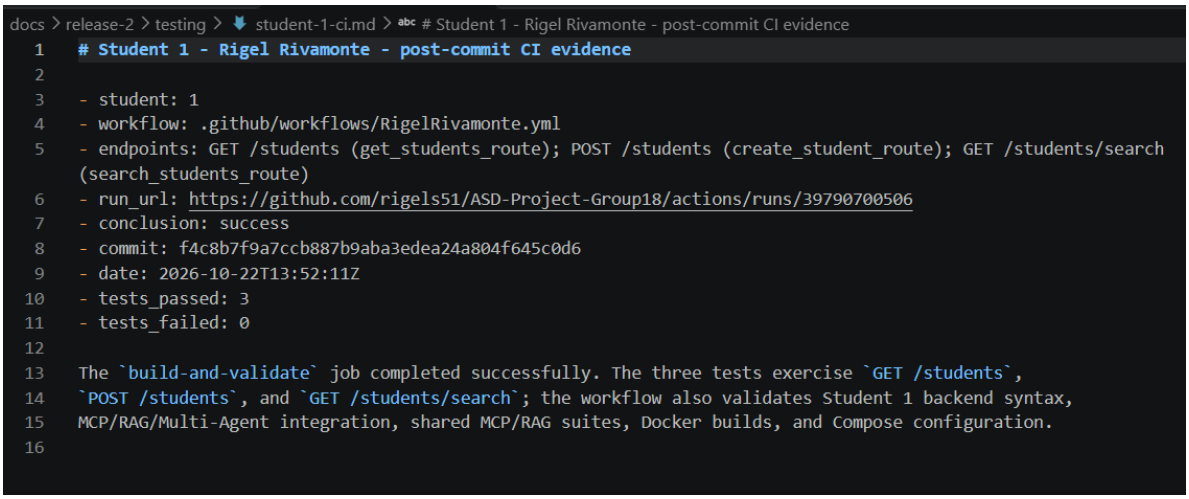
Ruff security rules (flake8-bandit S).....Passed
detect-secrets (no new credentials).....Passed
pip-audit (known-vulnerable dependencies).....Passed
```

Vu Tien Thanh Nguyen



6.0 Post-Commit CI/CD Testing

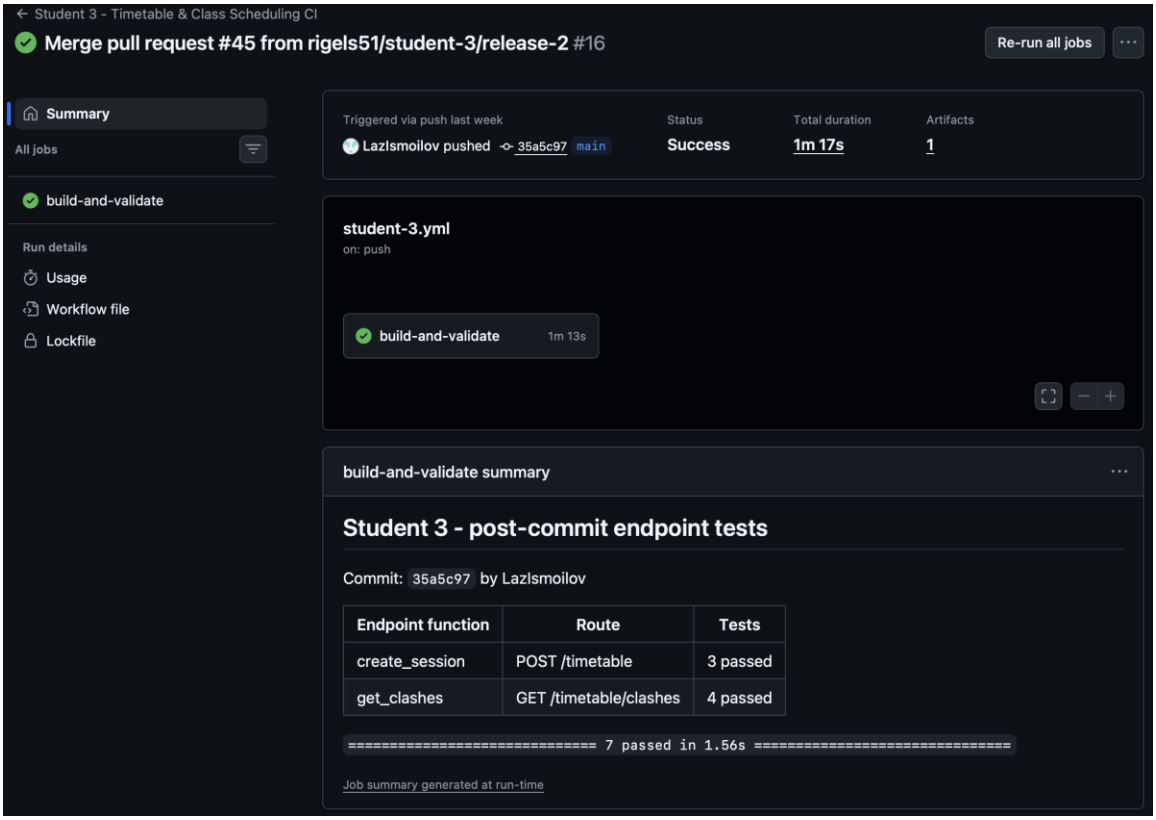
Rigel Rivamonte



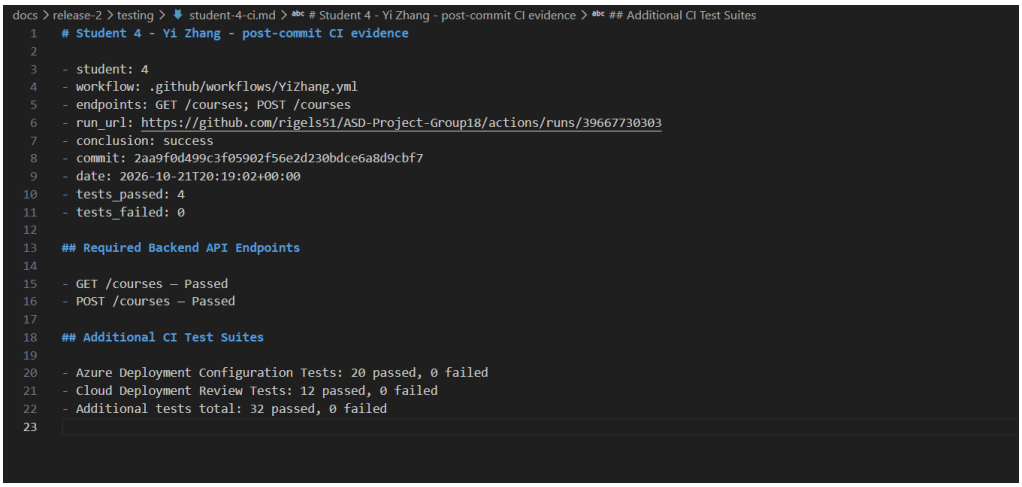
Jeriko Arceo



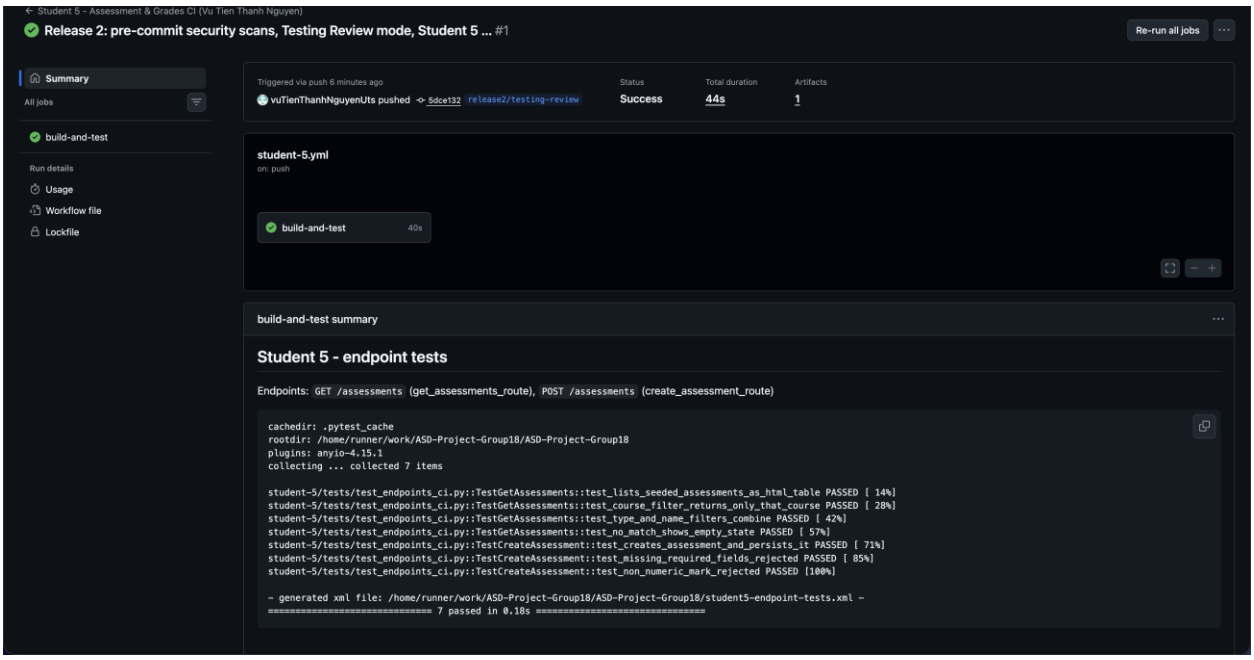
Lazizbek Ismoilov

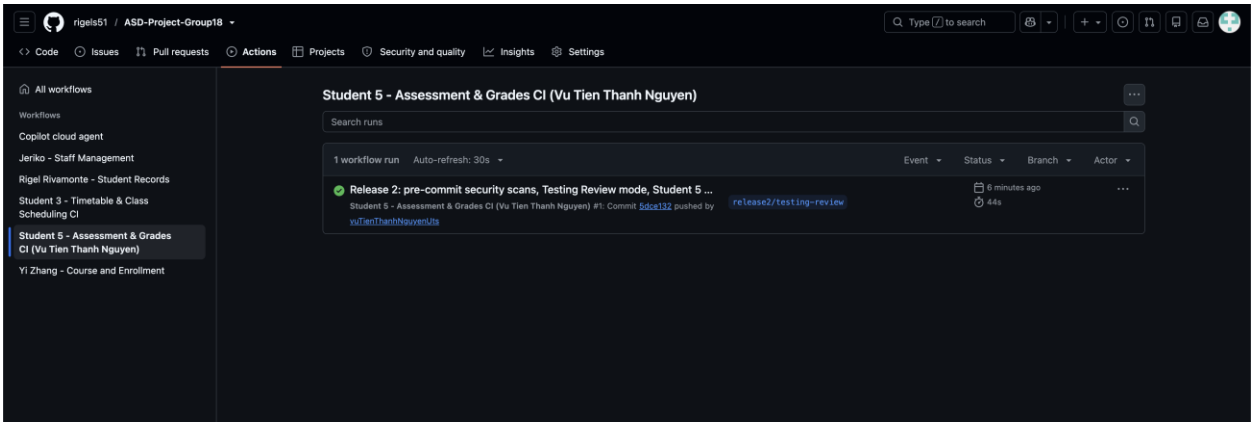


Yi Zhang



Vu Tien Thanh Nguyen





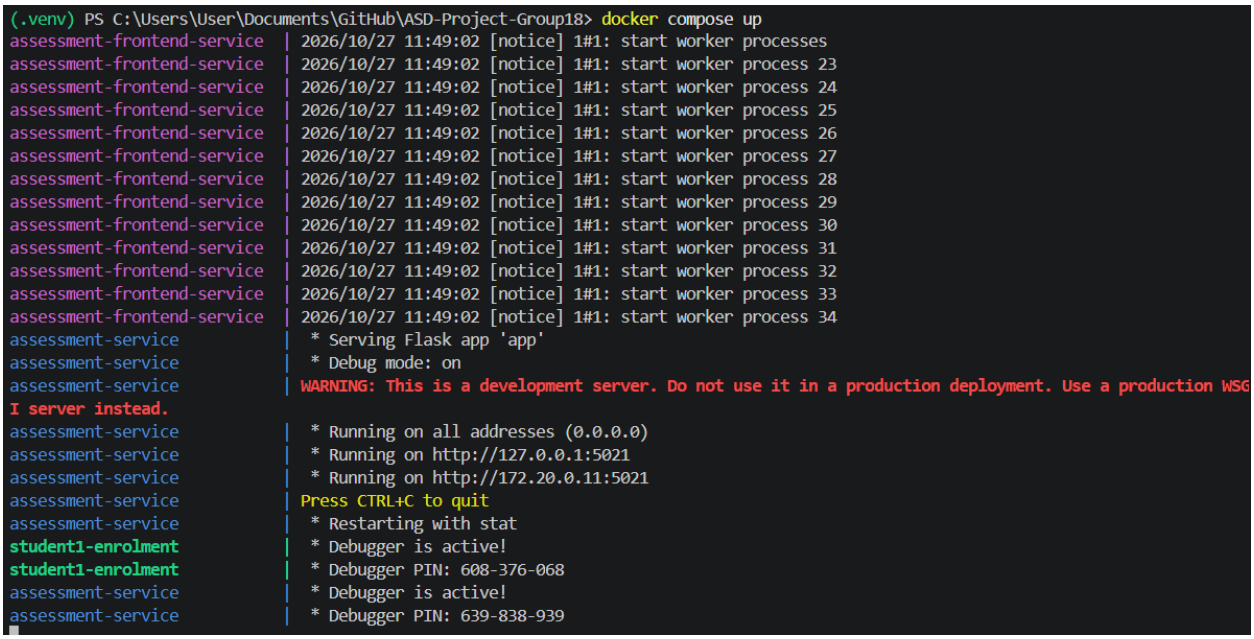
7.0 Testing Review

8.0 Local Deployment Evidence

```
Requirement already satisfied: flask==3.1.3 in .\.venv\Lib\site-packages (from -r ai-services\multi-agent-server\requirements.txt (line 1)
Requirement already satisfied: sniffio in .\.venv\Lib\site-packages (from openai->-r ai-services\multi-agent-server\requirements.txt (line
 3)) (1.3.1)
Requirement already satisfied: typing-extensions<5,>=4.14 in .\.venv\Lib\site-packages (from openai->-r ai-services\multi-agent-server\req
uirements.txt (line 3)) (4.16.0)
Requirement already satisfied: httpcore2==2.12.0 in .\.venv\Lib\site-packages (from httpx2<3,>=2.7.0->openai->-r ai-services\multi-agent-s
erver\requirements.txt (line 3)) (2.12.0)
Requirement already satisfied: truststore>=0.10 in .\.venv\Lib\site-packages (from httpx2<3,>=2.7.0->openai->-r ai-services\multi-agent-se
rver\requirements.txt (line 3)) (0.10.4)
Requirement already satisfied: h11>=0.16 in .\.venv\Lib\site-packages (from httpcore2==2.12.0->httpx2<3,>=2.7.0->openai->-r ai-services\mu
lti-agent-server\requirements.txt (line 3)) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in .\.venv\Lib\site-packages (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10
.13->openai->-r ai-services\multi-agent-server\requirements.txt (line 3)) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in .\.venv\Lib\site-packages (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.
13->openai->-r ai-services\multi-agent-server\requirements.txt (line 3)) (2.46.5)
Requirement already satisfied: typing-inspection>=0.4.2 in .\.venv\Lib\site-packages (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.
10.13->openai->-r ai-services\multi-agent-server\requirements.txt (line 3)) (0.4.4)

[notice] A new release of pip is available: 26.2.1 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> cd ai-services\multi-agent-server
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18\ai-services\multi-agent-server> python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5004
 * Running on http://192.168.0.15:5004
Press CTRL+C to quit
```

```
 (.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> docker compose build
  => => exporting layers                                                                                    6.6s
  => => exporting manifest sha256:d7e0062785321be82b84b295a80c15916d4bf1dd07f16b96f40b2b1b631ef839           0.1s
  => => exporting config sha256:7eb954b9f45591b08d3940efa4fc5ab92c15f271b428282ae0660382fc42a4ff            0.1s
  => => exporting attestation manifest sha256:a24d4862747dd156d6579ed1c09b26c3d674c5ced8ab491adeaa9c818ba9cc9a  0.1s
  => => exporting manifest list sha256:3e547de26a9c5087bfce363958a830ab43c323b1c87b2571e488da67094dcf82     0.1s
  => => naming to docker.io/library/asd-project-group18-assessment-service:latest                           0.0s
  => => unpacking to docker.io/library/asd-project-group18-assessment-service:latest                        3.1s
  => [timetable-backend] resolving provenance for metadata file                                             0.1s
  => [student1-enrolment] resolving provenance for metadata file                                            0.0s
  => [student4-enrolment] resolving provenance for metadata file                                            0.0s
  => [assessment-service] resolving provenance for metadata file                                            0.1s
  => [student2-backend] resolving provenance for metadata file                                              0.0s
 [+] build 15/15
  ✓ Image asd-project-group18-assessment-frontend-service Built                                            68.9s
  ✓ Image asd-project-group18-assessment-service          Built                                            69.0s
  ✓ Image asd-project-group18-homepage                    Built                                            69.3s
  ✓ Image asd-project-group18-assessment-database-service Built                                            68.8s
  ✓ Image asd-project-group18-student4-database           Built                                            68.5s
  ✓ Image asd-project-group18-student1-database           Built                                            68.8s
  ✓ Image asd-project-group18-timetable-frontend          Built                                            69.2s
  ✓ Image asd-project-group18-student1-frontend           Built                                            68.7s
  ✓ Image asd-project-group18-student2-database           Built                                            69.0s
  ✓ Image asd-project-group18-student4-frontend           Built                                            68.7s
  ✓ Image asd-project-group18-timetable-backend           Built                                            69.4s
  ✓ Image asd-project-group18-student4-enrolment          Built                                            68.6s
  ✓ Image asd-project-group18-student2-frontend           Built                                            69.3s
  ✓ Image asd-project-group18-student1-enrolment          Built                                            69.2s
  ✓ Image asd-project-group18-student2-backend            Built                                            69.1s
○ (.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18>
```



9.0 Cloud Deployment Report and Review

9.1 Platform choice.

We chose Azure Container Apps over Amazon ECS/Fargate. It is the preferred option in the specification, gives every app a stable HTTPS address, and is covered by the Azure for Students credit without a credit card.

9.2 Deployment script execution evidence.

scripts/deploy/deploy_azure.py has four modes:

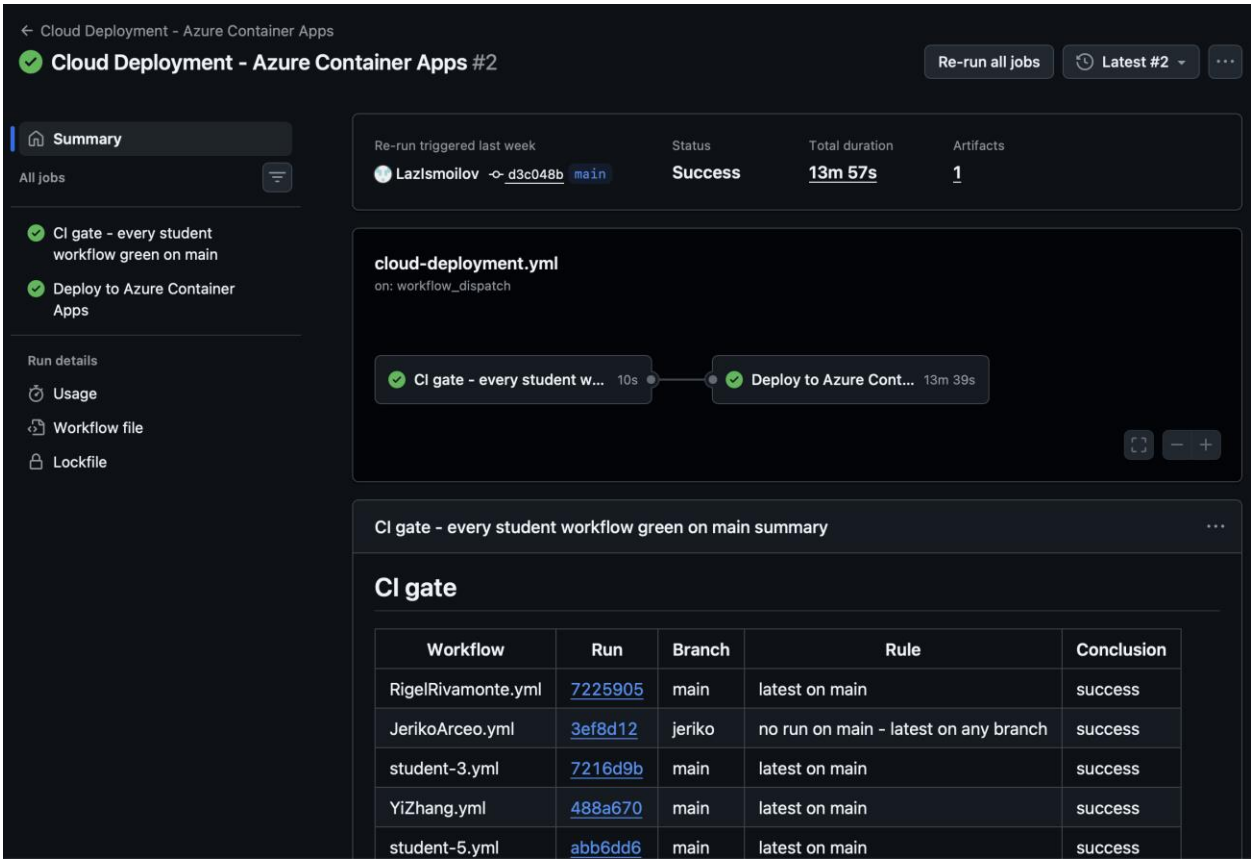
- plan: dry run that prints every command
- deploy (with --only and --skip-build for partial redeploys)
- status
- destroy –yes

A deployment creates all 15 apps, smoke-tests every public frontend, backend and the homepage, and writes docs/release-2/cloud/deployment.json. In the final run, every one returned HTTP 200.

```
homepage/ -> 200
student1-frontend/ -> 200
student2-frontend/ -> 200
timetable-frontend/ -> 200
student4-frontend/ -> 200
assessment-frontend-service/ -> 200
student1-enrolment/students -> 200
student2-backend/staff -> 200
timetable-backend/timetable -> 200
student4-enrolment/courses -> 200
assessment-service/assessments -> 200
(base) laz@Lazizbeks-MacBook-Air ASD-Project-Group18 %
```

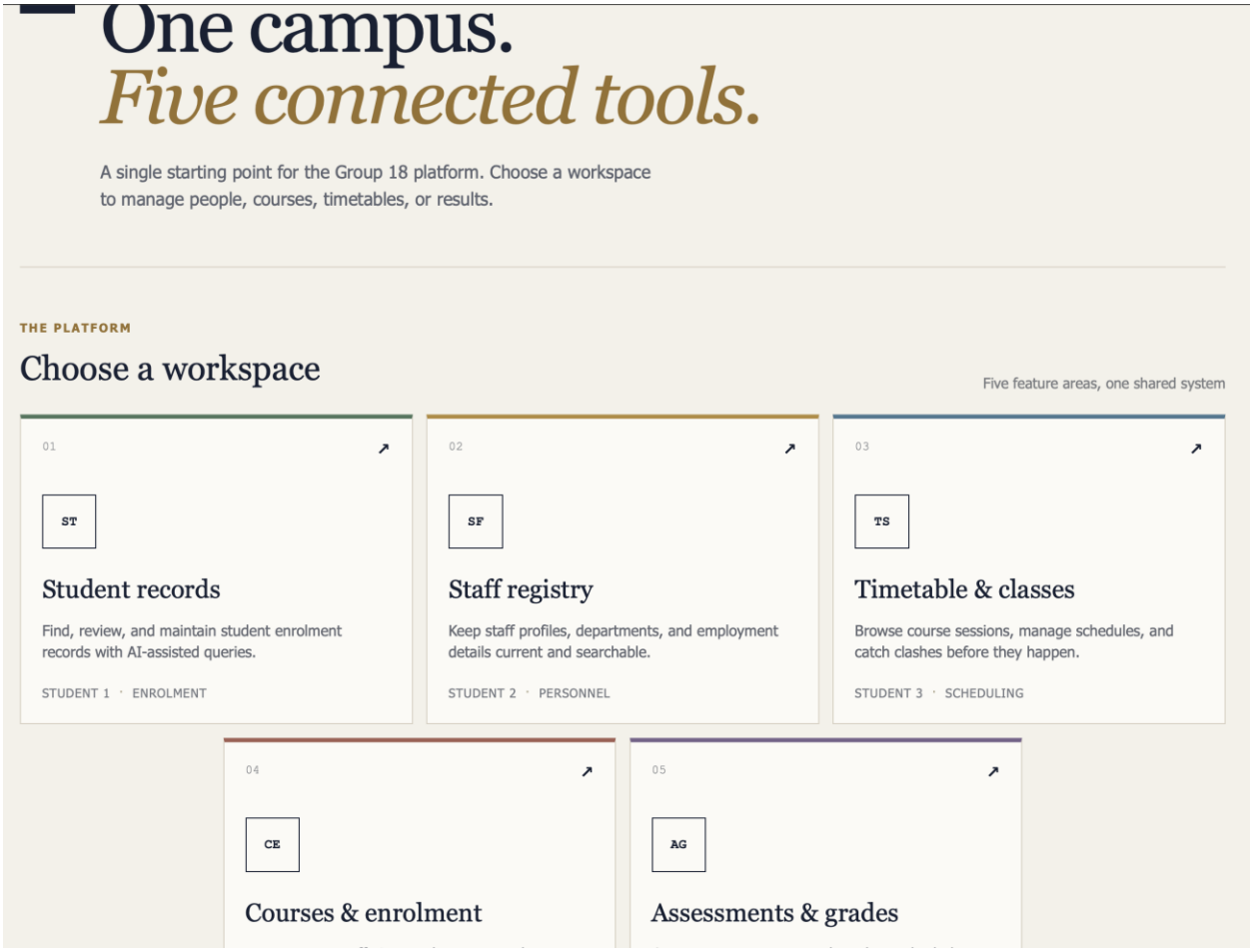
9.3 GitHub Actions deployment evidence.

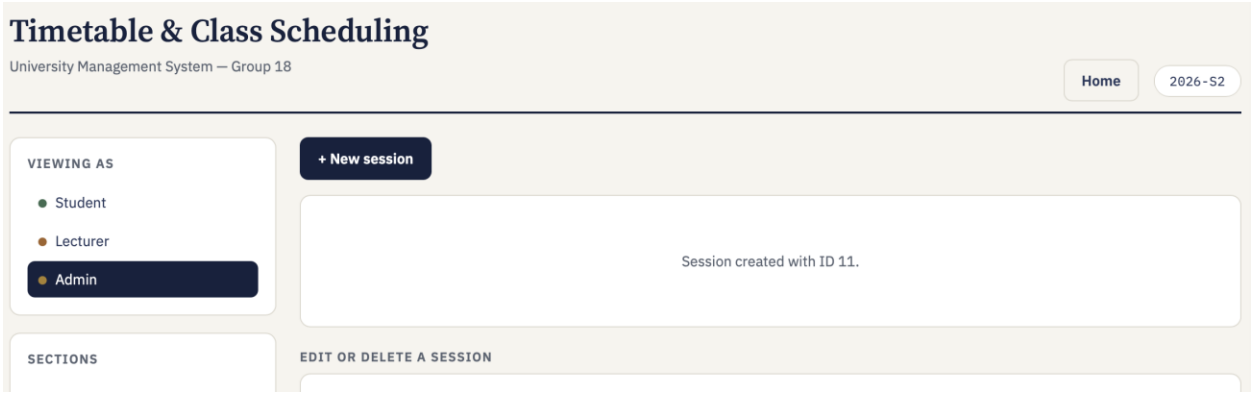
Run 39620962050 of cloud-deployment.yml passed the CI gate (all five student workflows green) and the Azure deploy job.



9.4 Live cloud evidence.

The application is live at https://homepage.victoriouswater-56424206.australiaeast.azurecontainerapps.io. Every feature loads from the homepage. Timetable create, read and clash detection work against the cloud database.





Timetable & Class Scheduling

University Management System — Group 18

Home 2026-S2

VIEWING AS

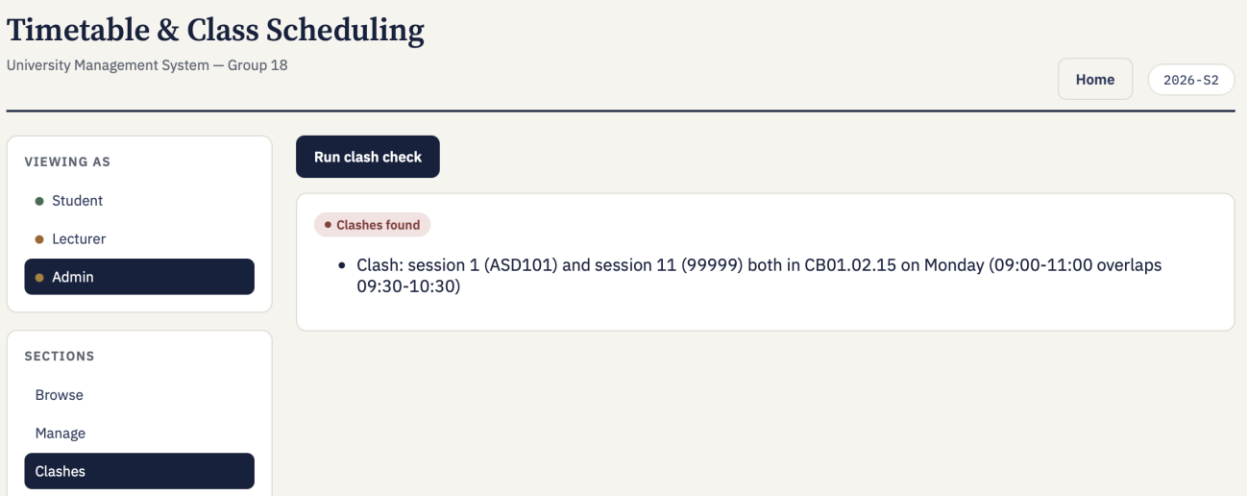
- Student
- Lecturer
- Admin

SECTIONS

- Browse
- Manage
- Clashes
- Ask AI
- MCP Tools
- Policy Q&A
- Multi-Agent

Filter by course code, e.g. ASD101 | Filter | View all sessions

ID	COURSE	TYPE	DAY	START	END	ROOM	SEMESTER
1	ASD101	Lecture	Monday	09:00	11:00	CB01.02.15	2026-S2
2	ASD101	Tutorial	Monday	11:00	12:00	CB01.02.16	2026-S2
3	WEB201	Lecture	Tuesday	09:00	11:00	CB01.03.01	2026-S2
4	WEB201	Lab	Tuesday	13:00	15:00	CB01.03.02	2026-S2
5	DBS101	Lecture	Wednesday	10:00	12:00	CB02.01.10	2026-S2
6	DBS101	Tutorial	Wednesday	13:00	14:00	CB02.01.11	2026-S2
7	NET201	Lecture	Thursday	09:00	11:00	CB02.02.05	2026-S2
8	NET201	Lab	Thursday	14:00	16:00	CB02.02.06	2026-S2
9	SEC301	Lecture	Friday	09:00	11:00	CB03.01.01	2026-S2
10	SEC301	Tutorial	Friday	11:00	12:00	CB03.01.02	2026-S2
11	99999	Lecture	Monday	09:30	10:30	CB01.02.15	2026-S2



9.5 Cloud Deployment Review (agentic loop).

I added a --mode cloud review to the shared Plan → Act → Observe → Adapt loop. In the Plan and Act steps, it inspects cloud-deployment.yml and the deploy script, probes every live frontend and backend, and checks that each MCP, RAG and Multi-Agent route answers “disabled”. In the Observe step, all 25 checks passed and the loop recommended **Release**. In the Adapt step, I recorded the human decision **Release**, which agreed with the recommendation. The decision is logged to docs/release-2/agentic-loop/cloud-release-decisions.jsonl, and the full report is saved to docs/release-2/agentic-loop/cloud-validation.md. In an earlier run, the review agent recommended “Do not release” because it read the disabled AI routes as faults. This shows why a human makes the final release decision.

```
OBSERVE: 25 pass, 0 warn, 0 missing, 0 fail. Backend read latency: max 1835 ms, avg 1225 ms. Recommended release decision: Release.
  Owner                        | Check                            | Result | Detail

  -----------------------------+----------------------------------+--------+-------------------------------------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------
  group                        | workflow cloud-deployment.yml    | PASS   | platform=Azure, gated on CI=True, AI flags off=3/3, credentials from secrets=True

  group                        | deploy scripts                   | PASS   | README.md, __init__.py, deploy_azure.py, test_deploy_azure.py

  group                        | deployment record                | PASS   | Azure Container Apps (australiaeast), commit d3c048b, 2026-10-21T15:09:45+00:00, run https://github.com/rige
ls51/ASD-Project-Group18/actions/runs/39620962050
  homepage                     | frontend loads                   | PASS   | https://homepage.victoriouswater-56424206.australiaeast.azurecontainerapps.io - HTTP 200 in 653 ms

  student1-frontend            | frontend loads                   | PASS   | https://student1-frontend.victoriouswater-56424206.australiaeast.azurecontainerapps.io - HTTP 200 in 610 ms

  student2-frontend            | frontend loads                   | PASS   | https://student2-frontend.victoriouswater-56424206.australiaeast.azurecontainerapps.io - HTTP 200 in 701 ms

  timetable-frontend           | frontend loads                   | PASS   | https://timetable-frontend.victoriouswater-56424206.australiaeast.azurecontainerapps.io - HTTP 200 in 607 ms

  student4-frontend            | frontend loads                   | PASS   | https://student4-frontend.victoriouswater-56424206.australiaeast.azurecontainerapps.io - HTTP 200 in 603 ms

  assessment-frontend          | frontend loads                   | PASS   | https://assessment-frontend-service.victoriouswater-56424206.australiaeast.azurecontainerapps.io - HTTP 200
in 612 ms
  S1 Rigel - Student Records   | GET /students                    | PASS   | HTTP 200 in 1223 ms

  S1 Rigel - Student Records   | POST /rag/answer disabled        | PASS   | HTTP 503 in 610 ms, response says disabled

  S2 Jeriko - Staff Management | GET /staff                       | PASS   | HTTP 200 in 1229 ms

  S2 Jeriko - Staff Management | POST /rag/answer disabled        | PASS   | HTTP 403 in 608 ms, response says disabled

  S3 Lazizbek - Timetable      | GET /timetable                   | PASS   | HTTP 200 in 611 ms

  S3 Lazizbek - Timetable      | POST /rag/answer disabled        | PASS   | HTTP 503 in 611 ms, response says disabled

  S3 Lazizbek - Timetable      | POST /mcp/call disabled          | PASS   | HTTP 503 in 609 ms, response says disabled

  S3 Lazizbek - Timetable      | POST /multi-agent/run disabled   | PASS   | HTTP 503 in 611 ms, response says disabled

  S4 Yi - Course & Enrollment  | GET /courses                     | PASS   | HTTP 200 in 1225 ms

  S4 Yi - Course & Enrollment  | POST /ask disabled               | PASS   | HTTP 503 in 610 ms, response says disabled

  S4 Yi - Course & Enrollment  | POST /mcp/run disabled           | PASS   | HTTP 503 in 610 ms, response says disabled

  S4 Yi - Course & Enrollment  | POST /rag/answer disabled        | PASS   | HTTP 503 in 609 ms, response says disabled

  S4 Yi - Course & Enrollment  | POST /multi-agent/run disabled   | PASS   | HTTP 503 in 611 ms, response says disabled

  S5 Vu - Assessment & Grades  | GET /assessments                 | PASS   | HTTP 200 in 1835 ms

  S5 Vu - Assessment & Grades  | POST /rag/ask disabled           | PASS   | HTTP 503 in 564 ms, response says disabled

  S5 Vu - Assessment & Grades  | POST /multi-agent/run disabled   | PASS   | HTTP 503 in 661 ms, response says disabled
```

```
  S3 Lazizbek - Timetable      | GET /timetable                   | PASS   | HTTP 200 in 611 ms

  S3 Lazizbek - Timetable      | POST /rag/answer disabled        | PASS   | HTTP 503 in 611 ms, response says disabled

  S3 Lazizbek - Timetable      | POST /mcp/call disabled          | PASS   | HTTP 503 in 609 ms, response says disabled

  S3 Lazizbek - Timetable      | POST /multi-agent/run disabled   | PASS   | HTTP 503 in 611 ms, response says disabled

  S4 Yi - Course & Enrollment  | GET /courses                     | PASS   | HTTP 200 in 1225 ms

  S4 Yi - Course & Enrollment  | POST /ask disabled               | PASS   | HTTP 503 in 610 ms, response says disabled

  S4 Yi - Course & Enrollment  | POST /mcp/run disabled           | PASS   | HTTP 503 in 610 ms, response says disabled

  S4 Yi - Course & Enrollment  | POST /rag/answer disabled        | PASS   | HTTP 503 in 609 ms, response says disabled

  S4 Yi - Course & Enrollment  | POST /multi-agent/run disabled   | PASS   | HTTP 503 in 611 ms, response says disabled

  S5 Vu - Assessment & Grades  | GET /assessments                 | PASS   | HTTP 200 in 1835 ms

  S5 Vu - Assessment & Grades  | POST /rag/ask disabled           | PASS   | HTTP 503 in 564 ms, response says disabled

  S5 Vu - Assessment & Grades  | POST /multi-agent/run disabled   | PASS   | HTTP 503 in 661 ms, response says disabled


ADAPT:
  Every cloud check passed; no rule-based adaptation needed.
  Implementation agent (qwen2.5:0.5b):
  The deployed application is usable. The AI routes (RAG, MCP, Multi-Agent) are disabled, and the rule-based release recommendation aligns with the baseline cloud profile.
  Review agent (llama3.1:8b):
  Risk: low - The deployment meets all requirements and has been thoroughly tested.

  Correction: None

  Retest: Not required as all checks have passed.

  Recommendation: Release - All tests have passed, including frontend loads and API calls, with AI routes disabled as per the baseline cloud profile.

HUMAN RELEASE DECISION: Loop recommendation: Release.
```

```
HUMAN RELEASE DECISION (agentic loop recommends: Release)
  1 - Release
  2 - Release with conditions
  3 - Do not release
Decision (1-3, Enter to leave pending): 1
Your name: Lazizbek
Reason: Release. All checks pass. The agent flagged the disabled MCP, RAG and Multi-Agent routes, but disabled is the require
d cloud configuration.
  Release by Lazizbek at 2026-10-27T15:15:48+00:00 (agrees with the loop recommendation).
  Reason: Release. All checks pass. The agent flagged the disabled MCP, RAG and Multi-Agent routes, but disabled is the requi
red cloud configuration.
  Logged to docs/release-2/agentic-loop/cloud-release-decisions.jsonl

Saved: docs/release-2/agentic-loop/cloud-validation.md

================================================================
SHARED AGENTIC LOOP COMPLETE
================================================================
(base) laz@Lazizbeks-MacBook-Air ASD-Project-Group18 %
```

9.6 Issues found and fixed during deployment

- Azure created an *express* environment, which blocks plain HTTP between apps. Internal certificate names also did not match, so service-to-service calls use HTTPS addresses.
- In-place updates did not restart containers. I confirmed this with curl, then changed the script to recreate apps on redeploy.
- The UTS tenant blocks service-principal creation. I replaced it with GitHub OIDC and a user-assigned managed identity.
- GitHub's OIDC token subject now uses immutable IDs, so I matched the federated credential to the exact subject format.
- The CI gate blocked deployment when a teammate's workflow had never run on main. I fixed the gate logic and documented it.

9.7 Known limitations

- Ollama is not hosted in the cloud, so AI-Mode does not answer there. MCP, RAG and Multi-Agent return "disabled" through their flags, but AI-Mode /ask fails instead of reporting "disabled". Hosting an approved model on Azure was outside the student credit budget.
- Because of the express environment, all 15 apps, including the four database services, have public HTTPS addresses. An IP allow-list or API key would close this.
- SQLite lives inside each container, so cloud data resets when an app is recreated.

10.0 Multi-Agent Workflow Evidence

10.1 Scenario:

Summarize the staff in the Arts department. List each person's name and employment type, count the full-time and part-time staff, and flag anything the records don't tell us, and don't include personal contact details.

STAFF MULTI-AGENT REVIEW

The Planner, Worker, and Reviewer prepare an evidence-based staff answer. You make the final decision.

Connected to shared Multi-Agent server. Domains: assessment, course, staff, students, timetable.

Question about staff records

Summarize the staff in the Arts department. List each person's name and employment type, count the full-time and part-time staff, and flag anything the records don't tell us, and don't include personal contact details.

Run workflow

1. PLANNER

```
Objective: Summarize the staff in the Arts department.
Steps: 1. Filter the staff data to only include those from the Arts department.
2. Count the full-time and part-time staff.
3. Flag any records that don't provide the required information.
4. Summarize the staff in the Arts department.
Evidence Required: None.
Human Approval: Required before the output is used.
```

2. WORKER (3 EVIDENCE RECORDS)

```
Output: 3 staff member(s) matching Arts department and Full-time employment and Part-time employment:
- Nero Garcia
- Tiffany Day
- Jane Remover
Evidence Used: staff service
Limitations: Exact match against retrieved staff records.
```

3. REVIEWER

Risk: low · Recommendation: Accept

```
Risk: Low - The worker's output appears to be accurate and complete, but the evidence sample only includes three staff members,
which may not be representative of the entire Arts department.

Correction: None

Retest: Retest the worker's output against a larger sample of staff records to ensure accuracy and completeness.

Recommendation: Accept - The worker's output is generally accurate and complete, but further testing is recommended to confirm its
validity.
```

4. YOUR DECISION

Decision

Accept

Your name

Comment (required except for Accept)

Corrected output (required for Correct)

Record decision

STAFF MULTI-AGENT REVIEW

The Planner, Worker, and Reviewer prepare an evidence-based staff answer. You make the final decision.

Connected to shared Multi-Agent server. Domains: assessment, course, staff, students, timetable.

Question about staff records

Summarize the staff in the Arts department. List each person's name and employment type, count the full-time and part-time staff, and flag anything the records don't tell us, and don't include personal contact details.

Run workflow

1. PLANNER

```
Objective: Summarize the staff in the Arts department.
Steps: 1. Filter the staff data to only include those from the Arts department.
2. Count the full-time and part-time staff.
3. Flag any records that don't provide the required information.
4. Summarize the staff in the Arts department.
Evidence Required: None.
Human Approval: Required before the output is used.
```

2. WORKER (3 EVIDENCE RECORDS)

```
Output: 3 staff member(s) matching Arts department and Full-time employment and Part-time employment:
- Nero Garcia
- Tiffany Day
- Jane Remover
Evidence Used: staff service
Limitations: Exact match against retrieved staff records.
```

3. REVIEWER

Risk: low · Recommendation: Accept

```
Risk: Low - The worker's output appears to be accurate and complete, but the evidence sample only includes three staff members,
which may not be representative of the entire Arts department.

Correction: None

Retest: Retest the worker's output against a larger sample of staff records to ensure accuracy and completeness.

Recommendation: Accept - The worker's output is generally accurate and complete, but further testing is recommended to confirm its
validity.
```

4. YOUR DECISION

Decision

Accept

Your name

Jeriko

Comment (required except for Accept)

Corrected output (required for Correct)

Record decision

STAFF MULTI-AGENT REVIEW

The Planner, Worker, and Reviewer prepare an evidence-based staff answer. You make the final decision.

Connected to shared Multi-Agent server. Domains: assessment, course, staff, students, timetable.

Question about staff records

Summarize the staff in the Arts department. List each person's name and employment type, count the full-time and part-time staff, and flag anything the records don't tell us, and don't include personal contact details.

Run workflow

1. PLANNER

```
Objective: Summarize the staff in the Arts department.
Steps: 1. Filter the staff data to only include those from the Arts department.
2. Count the full-time and part-time staff.
3. Flag any records that don't provide the required information.
4. Summarize the staff in the Arts department.
Evidence Required: None.
Human Approval: Required before the output is used.
```

2. WORKER (3 EVIDENCE RECORDS)

```
Output: 3 staff member(s) matching Arts department and Full-time employment and Part-time employment:
- Nero Garcia
- Tiffany Day
- Jane Remover
Evidence Used: staff service
Limitations: Exact match against retrieved staff records.
```

3. REVIEWER

Risk: low · Recommendation: Accept

```
Risk: Low - The worker's output appears to be accurate and complete, but the evidence sample only includes three staff members,
which may not be representative of the entire Arts department.

Correction: None

Retest: Retest the worker's output against a larger sample of staff records to ensure accuracy and completeness.

Recommendation: Accept - The worker's output is generally accurate and complete, but further testing is recommended to confirm its
validity.
```

4. YOUR DECISION

Accept by Jeriko

FINAL OUTPUT

```
Output: 3 staff member(s) matching Arts department and Full-time employment and Part-time employment:
- Nero Garcia
- Tiffany Day
- Jane Remover
Evidence Used: staff service
Limitations: Exact match against retrieved staff records.
```

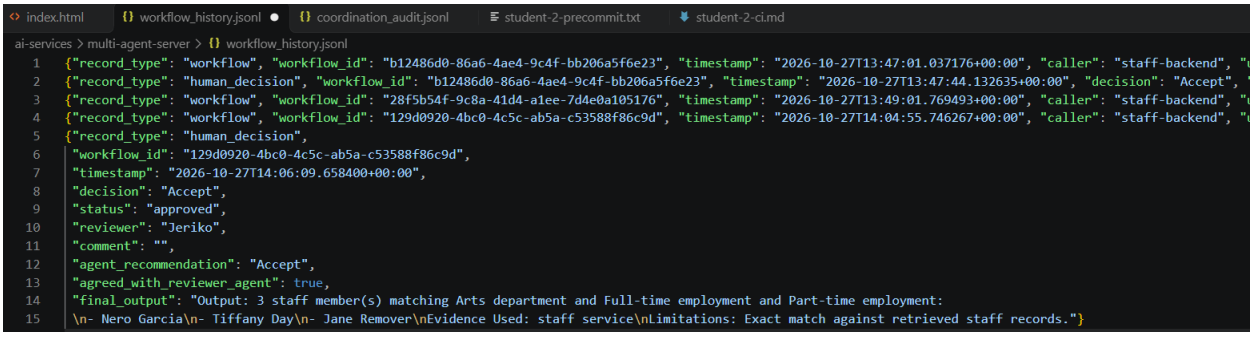
Workflow ID: 129d0920-4bc0-4c5c-ab5a-c53588f86c9d

RECENT STAFF WORKFLOWS

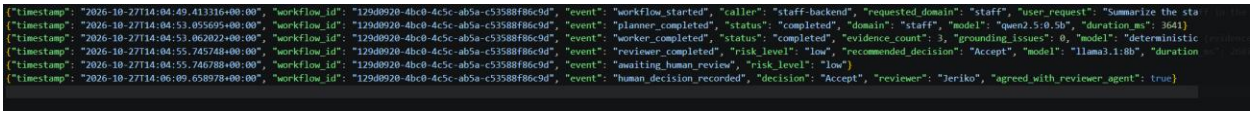
Refresh history

- 2026-10-27T14:04 · Summarize the staff in the Arts department. List each person's name and employment type, count the full-time and part-time staff, and flag anything the records don't tell us, and don't include personal contact details. · risk low · approved
- 2026-10-27T13:49 · What subject is Denver Mesa teaching? · risk medium · pending_human_review
- 2026-10-27T13:47 · Which staff are full time? · risk low · approved

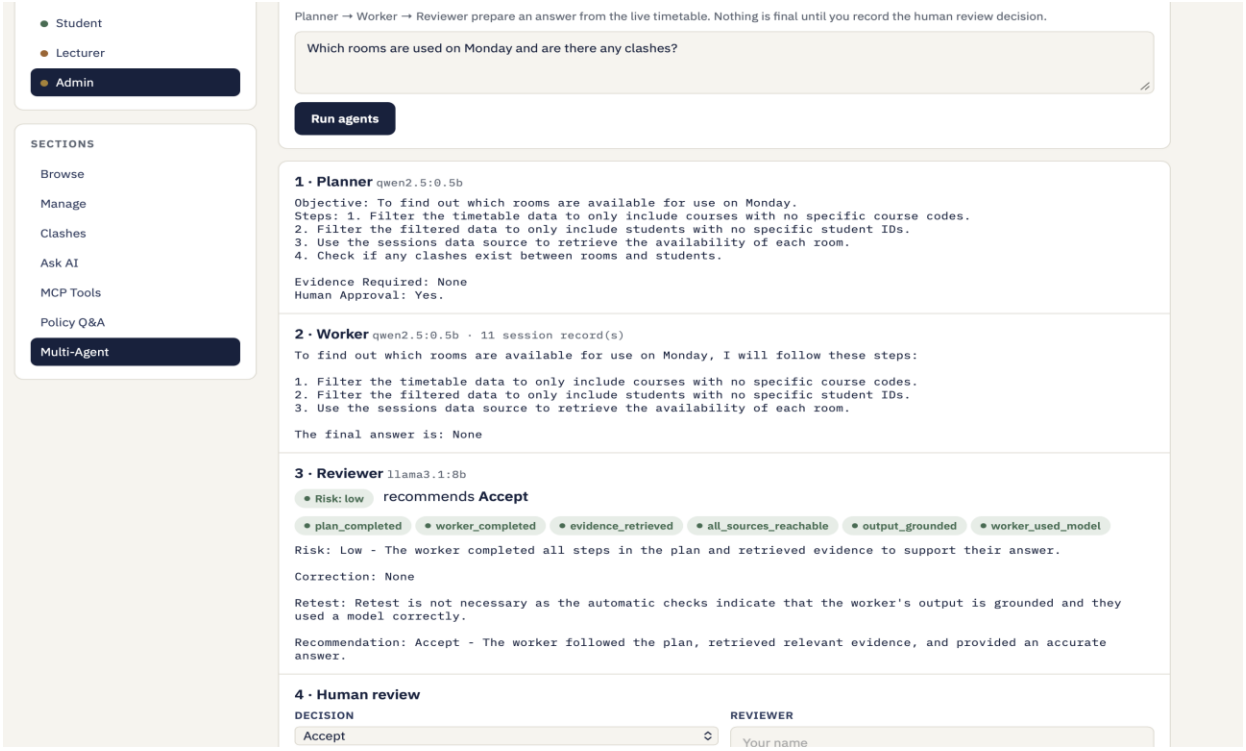
10.2 Workflow History



10.3 Coordination/Audit Log



10.4 Scenario: Timetable



4 · Human review

Correct by Lazizbek

Worker did not list the Monday sessions or identify the clash between sessions 1 and 11; corrected with the live timetable data.

```
Rooms used on Monday: CB01.02.15 and CB01.02.16.
    Sessions: ASD101 Lecture 09:00-11:00 CB01.02.15 (session 1); ASD101 Tutorial 11:00-12:00 CB01.02.16 (session
2); ASD101 Lecture 09:30-11:30 CB01.02.15 (session 11).
    Clash: session 1 and session 11 overlap in CB01.02.15 from 09:30 to 11:00. Session 2 does not clash, as it
starts when session 1 ends.
```

workflow 5dcc0ea3-0d15-4e8d-aa01-d4f47cd8fb1b

TIMETABLE WORKFLOW HISTORY

TIME (UTC)	REQUEST	RISK	STATUS
2026-10-27 15:32	Which rooms are used on Monday and are there any clashes?	low	corrected
2026-10-22 16:02	Summarise the timetable sessions for ASD101 and flag any room clashes.	low	corrected
2026-10-22 15:48	Which rooms are used on Monday and are there any clashes?	low	partially_approved
2026-10-22 15:45	Summarise the timetable sessions for ASD101 and flag any room clashes.	low	corrected
2026-10-20 13:33	Summarise the timetable sessions for ASD101.	low	approved
2026-10-20 13:33	Summarise the timetable sessions for ASD101 and flag any room clashes	low	corrected

```
(base) laz@Lazizbeks-MacBook-Air multi-agent-server % clear && ~/.venvs/multi-agent-server/bin/python workflow_runner.
y --domain timetable "Which rooms are used on Monday and are there any clashes?"

[1] PLANNER (qwen2.5:0.5b) - completed
Objective: To find out which rooms are available for use on Monday.
Steps: 1. Check the timetable to see if there's a room available for Monday.
2. Filter the results based on student IDs and course codes to exclude clashes.
Evidence Required: None required as no specific data sources were provided.
Human Approval: No human approval needed before using the output.

[2] WORKER (qwen2.5:0.5b) - completed, evidence=11
The rooms available for use on Monday are:

- CB01.02.15 (ASD101)
- CB01.03.01 (WEB201)
- CB02.01.10 (DBS101)
- CB02.01.11 (DBS101)
- CB02.02.05 (NET201)
- CB02.02.06 (NET201)
- CB03.01.01 (SEC301)

[3] REVIEWER (llama3.1:8b) - risk=medium, recommends Partially Accept
Risk: Medium - The output lists rooms that are available on Monday, but it also includes rooms with conflicting course
codes (e.g., CB02.01.10 and CB02.01.11 for DBS101).

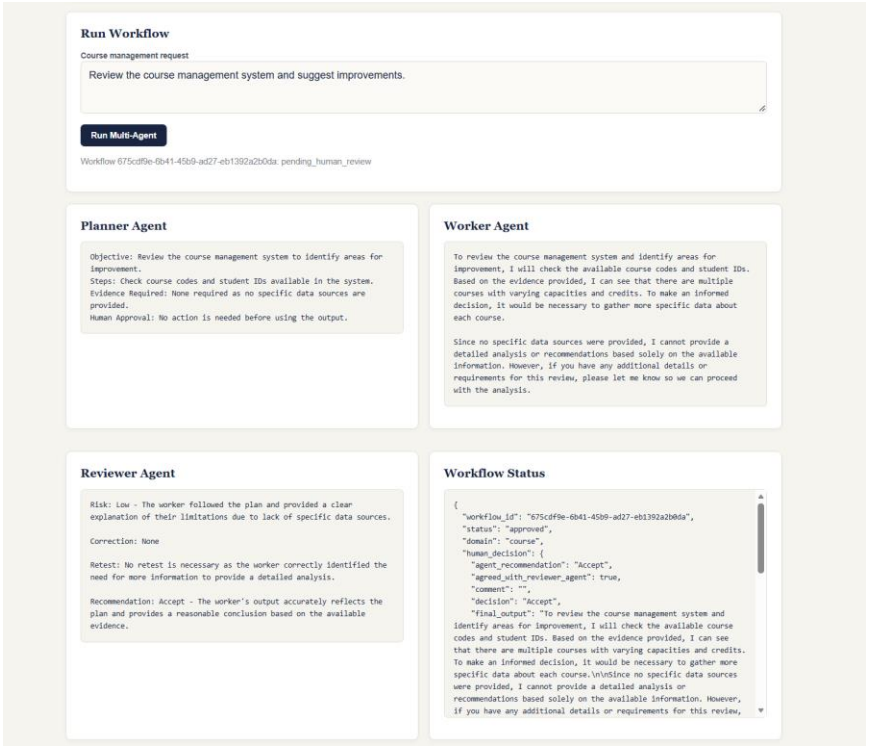
Correction: Remove the duplicate room entries and clarify which courses have clashes.

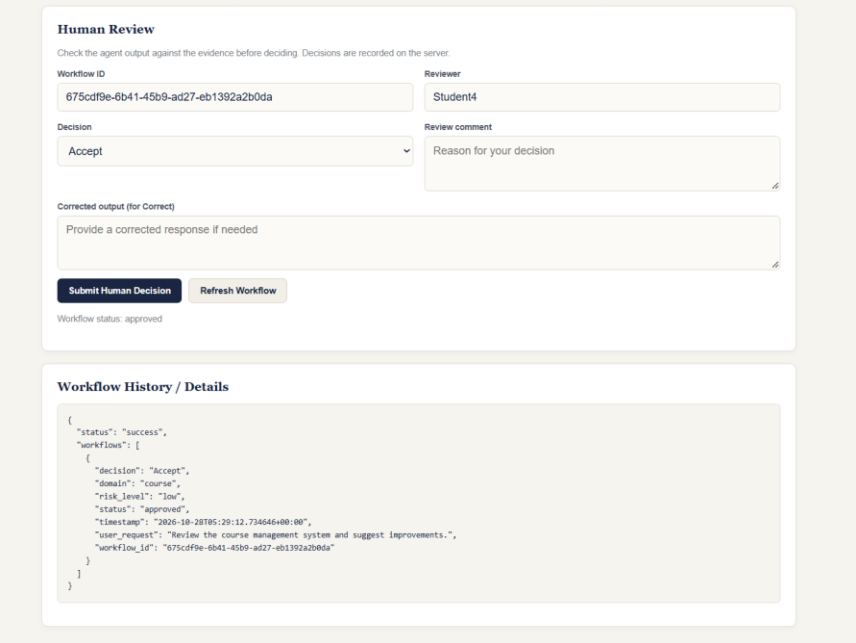
Retest: Verify the updated list of available rooms on Monday to ensure accuracy.

Recommendation: Partially Accept - The output partially meets the requirements, but requires further refinement to acc
rately identify available rooms without conflicts.
   checks: plan_completed=PASS, worker_completed=PASS, evidence_retrieved=PASS, all_sources_reachable=PASS, output_gro
nded=PASS, worker_used_model=PASS

[4] HUMAN REVIEW - Partially Accept by Lazizbek: Evidence is correct, but the Worker did not state the Monday rooms (C
01.02.15, CB01.02.16) or the clash between sessions 1 and 11 in CB01.02.15.
==========================================================================
```

10.5 Scenario: Course and Enrolment





10.6 Scenario: Student Records

```
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> $id = $r.workflow_id
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> "WORKFLOW ID: $id   caller=$($r.caller)   domain=$($r.domain)   status=$($r.status)"
WORKFLOW ID: a2c29d69-ad4d-4a68-a8bf-db734e2d9a98   caller=student1   domain=students   status=pending_human_review
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> "`n--- [1] PLANNER ---";  $r.planner_result.plan

--- [1] PLANNER ---
Objective: Find students in the Cybersecurity course.
Steps: 1. Filter students by course code to include only those in the Cybersecurity course.
Evidence Required: None.
Human Approval: Required before the output is used.
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> "`n--- [2] WORKER (evidence=$($r.worker_result.evidence_count)) ---"; $r.worker_result.output

--- [2] WORKER (evidence=12) ---
The students in the Cybersecurity course are as follows:

1. [STU-1003, STU-1008, STU-1009, STU-1011, STU-1010]
2. [STU-1005, STU-1006, STU-1007, STU-1004, STU-1002]
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> "`n--- [3] REVIEWER (risk=$($r.reviewer_result.risk_level), recommends $($r.reviewer_result.recommended_dec
sion)) ---"; $r.reviewer_result.review

--- [3] REVIEWER (risk=medium, recommends Partially Accept) ---
Risk: Medium - The output lists students in the Cybersecurity course, but the evidence provided includes students from other courses, which may indicate incomplete or ina
curate filtering.

Correction: The worker should re-filter the students to only include those with the correct course code, and verify that the output matches the plan.

Retest: Retest the worker with a revised plan that includes a clear course code filter.

Recommendation: Partially Accept - The output is partially correct, but requires further verification and correction to ensure accuracy.
(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18> $d = Invoke-RestMethod -Method Post -Uri http://localhost:5001/multi-agent/decision -Body @{
>>   workflow_id = $id
>>   decision    = "Accept"
>>   reviewer    = "Rigel Rivamonte"
>>   comment     = "Correct: lists the Cybersecurity students STU-1003 Andres Villamor and STU-1009 Lazizbek Ismoilov."
>> }
>> "--- [4] HUMAN REVIEW ---"; $d.human_decision | Format-List decision, reviewer, comment, status
--- [4] HUMAN REVIEW ---


decision : Accept
reviewer : Rigel Rivamonte
comment  : Correct: lists the Cybersecurity students STU-1003 Andres Villamor and STU-1009 Lazizbek Ismoilov.
status   : approved
```

```
--- workflow_history.jsonl ---


record_type : workflow
timestamp   : 2026-10-28T09:46:25.625570+00:00
caller      : student1
domain      : students
status      : pending_human_review

record_type : human_decision
timestamp   : 2026-10-28T09:55:09.429135+00:00
status      : approved
decision    : Accept
reviewer    : Rigel Rivamonte
comment     : Correct: lists the Cybersecurity students STU-1003 Andres Villamor and STU-1009 Lazizbek Ismoilov.



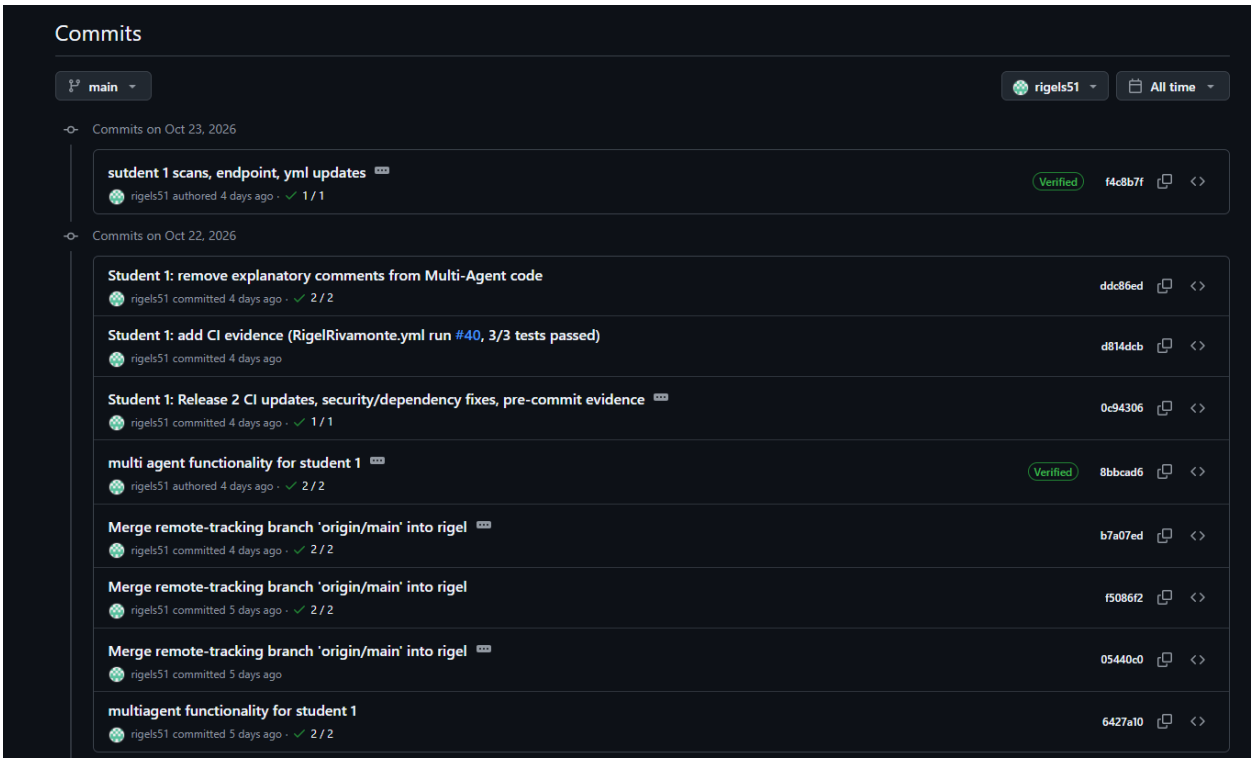
--- coordination_audit.jsonl ---

timestamp                       event                  status    risk_level decision reviewer
---------                       -----                  ------    ---------- -------- --------
2026-10-28T09:44:12.350104+00:00 workflow_started
2026-10-28T09:44:18.441375+00:00 planner_completed      completed
2026-10-28T09:44:32.408486+00:00 worker_completed       completed
2026-10-28T09:46:25.608675+00:00 reviewer_completed              medium
2026-10-28T09:46:25.632670+00:00 awaiting_human_review           medium
2026-10-28T09:55:09.431893+00:00 human_decision_recorded                    Accept   Rigel Rivamonte

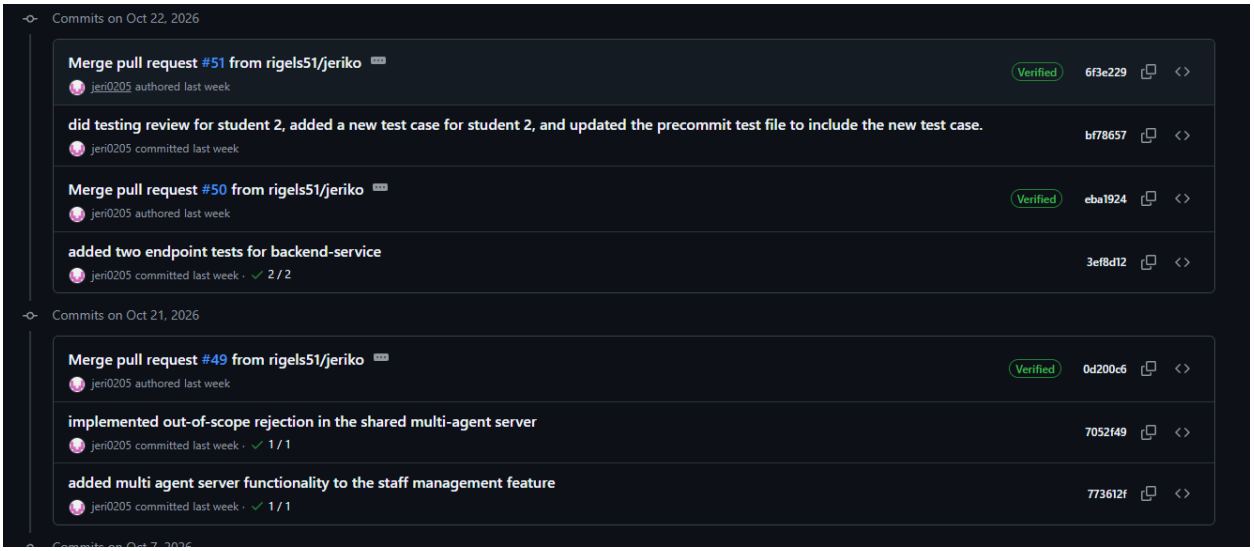

(.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18>
```

11.0 GitHub Commit Logs

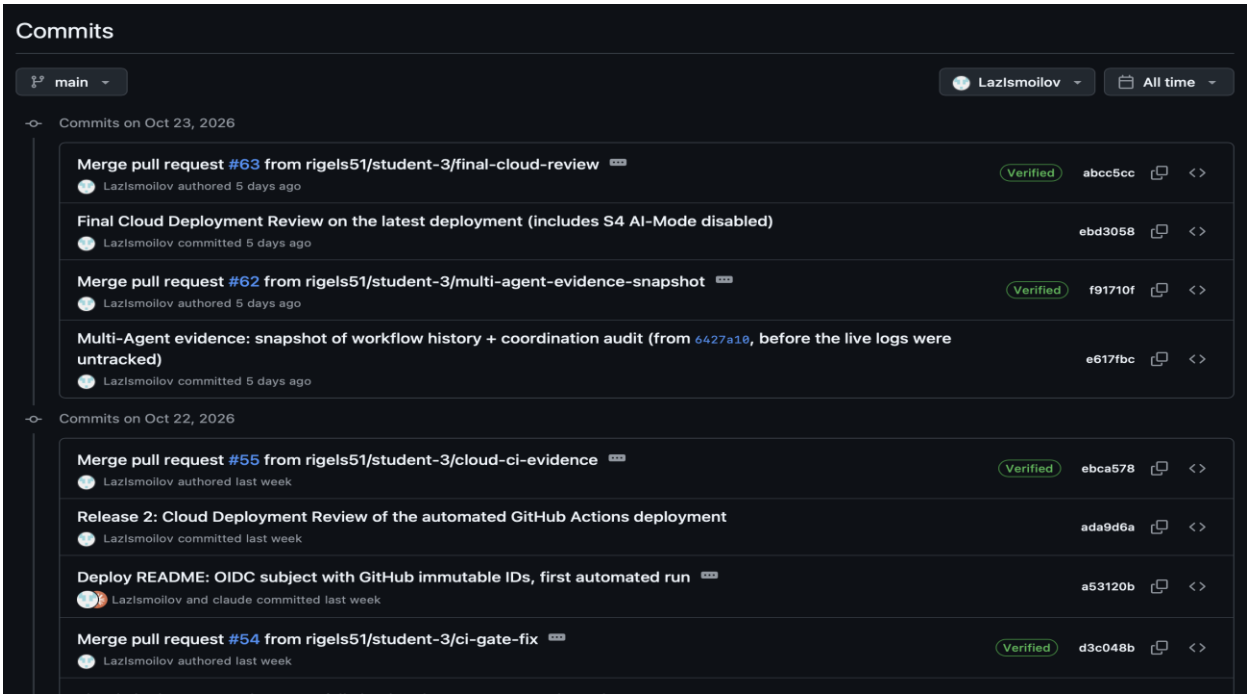
Rigel Rivamonte



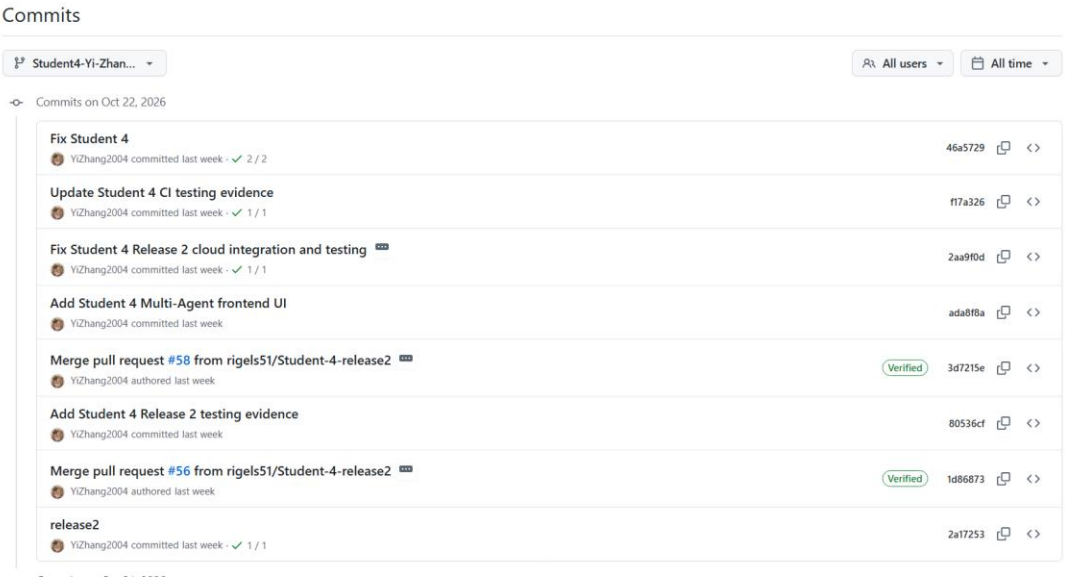
Jeriko Arceo



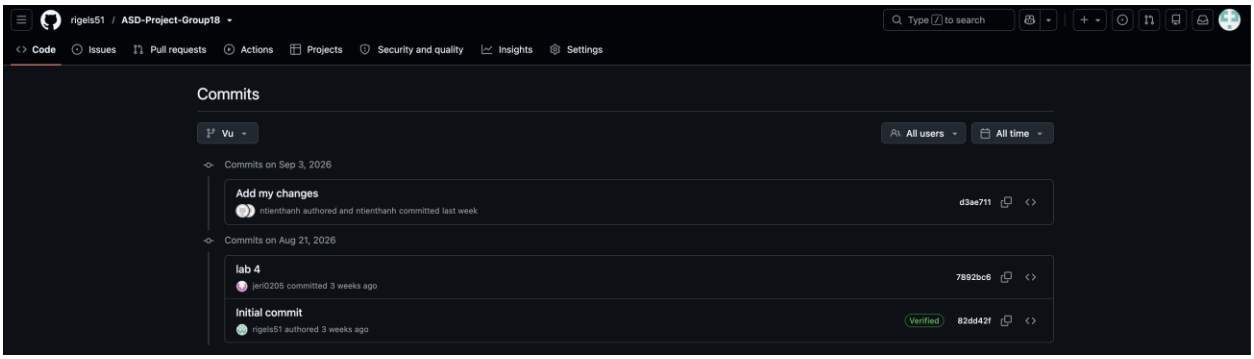
Lazizbek Ismoilov



Yi Zhang



Vu Tien Thanh Nguyen



12.0 Contribution Logs

Rigel Rivamonte

I integrated the Student Records feature with the existing shared Multi-Agent Server. This would include the backend routes for running workflows, recording human decisions, retrieving workflow details/history/status, the Multi-Agent service client, and the Student 1 frontend workflow and human-review interface. I also updated `RigelRivamonte.yml` for any CI workflow updates, endpoint testing, and Student 1 pre-commit and CI evidence.

Jeriko Arceo

For release 2, I connected the multi-agent server to the staff management feature which meant linking the planner, worker, and reviewer agents as well as the workflow history and coordination/audit logs to the front-end to allow users to access the tool. I also added out-of-scope rejection to the multi-agent server, ensuring that non-relevant queries are not made by the server.

Lazizbek Ismoilov

I integrated Timetable with the shared Multi-Agent Server: backend routes, the Multi-Agent tab with human review, and history. Fixed the duplicate-run guard and the Reviewer badge. Also, I wrote 7 post-commit endpoint tests and extended student-3.yml with the pytest stage, JUnit artifact and live clash check. I fixed the Flask debug-mode security finding. Pre-commit passes 3/3. I built the Cloud Deployment Review mode of the shared agentic loop as well. I also took ownership of the team cloud deployment: wrote deploy_azure.py and cloud-deployment.yml, set up OIDC authentication, and deployed all 15 services to Azure Container Apps. Furthermore, I fixed the CI gate for a teammate's workflow and recovered untracked Multi-Agent evidence logs.

Yi Zhang

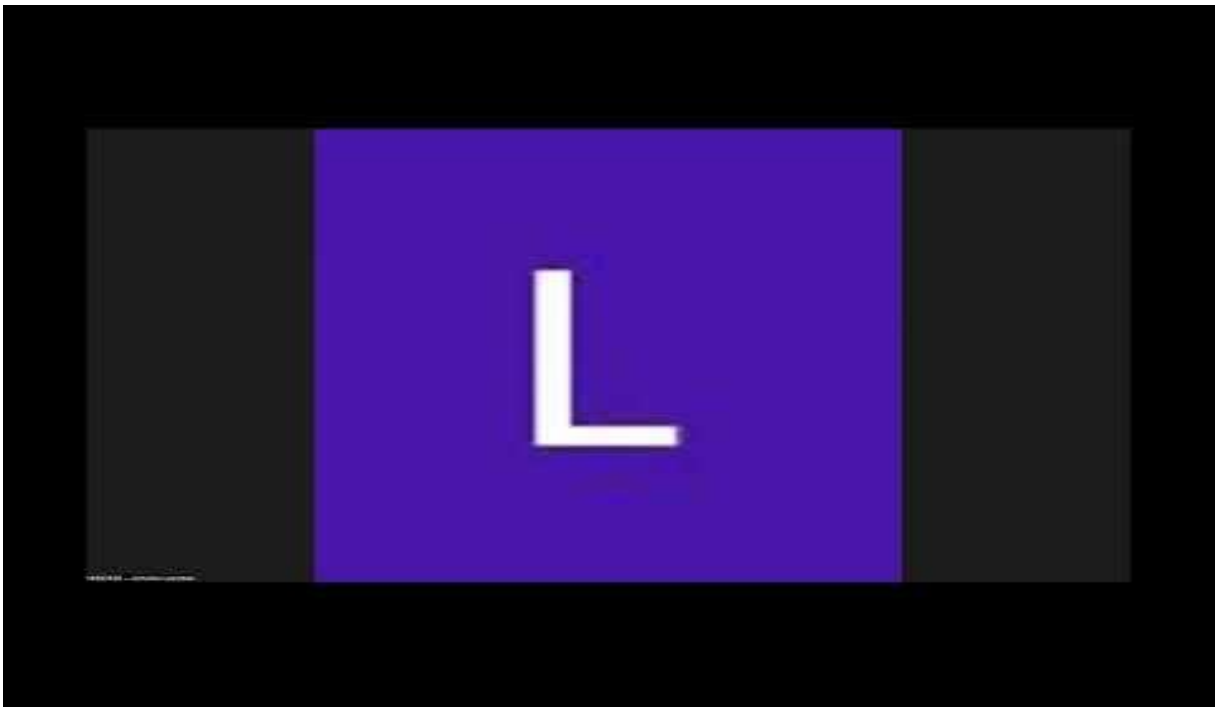
I integrated the Student 4 Course and Enrollment Management feature with the shared Multi-Agent Server, including Planner, Worker, Reviewer, Human Review, and workflow history. I also added automated API tests and updated GitHub Actions. Pre-commit security checks passed using Ruff, detect-secrets, and pip-audit. I also updated Student 4-related cloud configurations and helped integrate the feature into Azure deployment.

Vu Tien Thanh Nguyen

I implemented the shared Multi-Agent Server and integrated it with the Assessment and Grades Management feature, enabling the feature to use the Planner, Worker, and Reviewer agents, as well as human review and workflow history. I also implemented pre-commit checks for the project to improve code quality and security, and added post-commit automated tests for my feature to verify the backend endpoints and ensure the functionality works as expected.

13.0 Showcase Video

https://youtu.be/X2a1ERf_kUA


L